

15B17CI371 – Data Structures Lab
ODD 2026
Lab Test-1 [CO: C270.1, C270.2]

Max Marks: 20 Max Time: 50 minutes.

Instructions:

All students must save their programs using the following naming convention:
(Enroll.No_LT1_SetODD/EVEN) with docx or pdf formats.

For each question, mention the question, its solution C++ program, and its execution output. **Code of Conduct:**

1. **DO NOT use mobile phones, earphones, or any other electronic device during the during lab tests/evaluations.** Defaulters will receive zero marks in evaluations.
2. **AI Tool Usage Policy:** Zero marks will be awarded usage of AI tools during tests/evaluations.
3. Make a local copy of your question paper. **Disconnect the LAN** cord before starting your evaluation/test. Ensure that you save your work regularly on CodeBlocks.
4. Submit your solution after your evaluation is complete as per the instructions from your evaluators.
5. Any form of cheating during the Evaluation/test will result in awarding of 0 marks.
6. STL usage is NOT allowed.

Set: EVEN

Q1. An IT company assigns employees to multiple projects. Each employee can work on several projects, and each project can have multiple employees. Implement a **multilist** to maintain employee-project relationships.

The system should support:

- **Add Employee**
- **Add Project**
- **Assign Employee** to a project
- **Remove Employee** from a project
- **Display Projects of Employee**
- **Display Employees of Project**

Test Case

Input:

```
addEmployee(101, "Amit")
addEmployee(102, "Riya")
addEmployee(103, "Karan")
```

```
addProject(1, "Website")
addProject(2, "MobileApp")
addProject(3, "AIProject")
```

```
assign(101, 1)
assign(101, 2)
assign(102, 1)
assign(102, 3)
assign(103, 2)
```

```
displayProjects(101)
displayEmployees(1)
```

Output:

Projects of Employee 101:
Website → MobileApp

Employees working on Website:
101 → 102

Q2. An airline maintains passenger records in a **singly linked list**. Each record contains **passenger_id**, **passenger_name**, and **seat_no**. Before boarding, passengers need to be arranged according to their seat numbers. Implement **Merge Sort** to sort the passenger linked list in ascending order of **seat_no**. Display the records before and after sorting.

Test Case:

Passenger Records:

501 Amit 23
502 Ravi 11
503 Neha 35
504 Simran 18
505 Karan 7

Expected Output:

Before Sorting:
501 Amit 23
502 Ravi 11
503 Neha 35

504 Simran 18

505 Karan 7

After Sorting by Seat Number:

505 Karan 7

502 Ravi 11

504 Simran

18 501 Amit

23 503 Neha

35